Automating Employee Data

Project Documentation

Prepared by:

Team 4

1. Introduction

Project Title: Automating Employee Data

Team Members:

Name	Role	Responsibilities
Project Lead	Full Stack Developer	Architecture, integration, delivery
Backend Developer	Node.js / Express	API design, database management
Frontend Developer	React.js	UI/UX, dashboard components
QA Engineer	Testing	Unit, integration & UAT testing
DevOps Engineer	Infrastructure	Deployment, CI/CD pipeline

Version: 1.0

Date: 2025

2. Project Overview

Purpose

The Automating Employee Data project is a full-stack MERN application designed to digitize, centralize, and automate the management of employee records within an organization. The system eliminates manual HR data entry, reduces errors, and provides real-time access to employee information through an intuitive web interface.

Features

- Automated employee onboarding and offboarding workflows
- Centralized employee data management (personal info, job details, documents)
- Role-based access control for HR, managers, and employees
- Real-time dashboard with analytics and reporting
- Automated notifications and alerts for key HR events
- Bulk import/export of employee records via CSV/Excel
- Audit trail and activity log for compliance
- Integration with payroll and attendance systems

3. Architecture

Figure 1: System Architecture Overview

Frontend (React.js)

- Built with React.js 18 with functional components and hooks
- State management using Redux Toolkit
- UI components from Material-UI (MUI) library
- Axios for HTTP requests to the backend API
- React Router for client-side routing
- Chart.js / Recharts for data visualization and dashboards

Backend (Node.js & Express.js)

- RESTful API built with Node.js and Express.js
- JWT-based authentication and role-based authorization middleware
- Mongoose ODM for MongoDB interactions
- Multer for file upload handling (employee documents)
- Nodemailer for automated email notifications
- Express-validator for input validation and sanitization

Database (MongoDB)

- MongoDB Atlas (cloud-hosted) for scalable storage
- Collections: employees, users, departments, roles, audit_logs
- Indexed fields on employeeId, email, departmentId for fast queries
- Mongoose schemas enforce data types and validation rules

4. Setup Instructions

Prerequisites

Software	Version	Purpose
Node.js	>= 18.x	Runtime environment
MongoDB	>= 6.0 (Atlas)	Database
npm	>= 9.x	Package manager
Git	Latest	Version control
VS Code	Latest (recommended)	Code editor

Installation

1. Clone the repository:

```
git clone https://github.com/your-org/automating-employee-data.git
cd automating-employee-data
```

2. Install backend dependencies:

```
cd server
npm install
```

3. Install frontend dependencies:

```
cd ../client
npm install
```

4. Configure environment variables — create a .env file in /server:

```
PORT=5000
MONGO_URI=mongodb+srv://<username>:<password>@cluster.mongodb.net/employee_db
JWT_SECRET=your_super_secret_key
JWT_EXPIRES_IN=7d
EMAIL_HOST=smtp.gmail.com
EMAIL_PORT=587
EMAIL_USER=your_email@gmail.com
EMAIL_PASS=your_app_password
```

5. Folder Structure

Client (React Frontend)

```
client/
├── public/
├── src/
│   ├── components/      # Reusable UI components
│   ├── pages/           # Page-level components
│   ├── redux/           # Redux store, slices, thunks
│   ├── services/        # Axios API service calls
│   ├── utils/           # Helper functions
│   ├── App.js
│   └── index.js
└── package.json
```

Server (Node.js Backend)

```
server/
├── config/          # DB connection, environment config
├── controllers/     # Route handler logic
├── middlewares/     # Auth, error handling, validation
├── models/          # Mongoose schemas
├── routes/          # Express route definitions
├── utils/           # Email, file helpers
├── uploads/         # Uploaded employee documents
├── server.js
└── package.json
```

6. Running the Application

Development Mode

Run frontend and backend concurrently using separate terminal windows:

Frontend

```
cd client
npm start
# Runs at http://localhost:3000
```

Backend

```
cd server
npm run dev
# Runs at http://localhost:5000 (nodemon)
```

Production Build

```
cd client && npm run build
cd ../server && npm start
```

7. API Documentation

Base URL: http://localhost:5000/api

Employee Endpoints

Method	Endpoint	Description	Auth Required
GET	/employees	Get all employees	Yes
GET	/employees/:id	Get employee by ID	Yes
POST	/employees	Create new employee	Admin
PUT	/employees/:id	Update employee record	Admin/Manager
DELETE	/employees/:id	Delete employee	Admin
POST	/employees/import	Bulk import from CSV	Admin

Authentication Endpoints

Method	Endpoint	Description
POST	/auth/login	User login, returns JWT token
POST	/auth/logout	Invalidate session
POST	/auth/refresh-token	Refresh JWT token
POST	/auth/forgot-password	Send password reset email

Example Response

```
{
"success": true,
"data": {
"employeeId": "EMP-0042",
"name": "Ravi Kumar",
"department": "Engineering",
"role": "Senior Developer",
"email": "ravi.kumar@company.com",
"joiningDate": "2023-06-15"
  }
}
```

8. Authentication

Strategy

The application uses JSON Web Tokens (JWT) for stateless authentication. On successful login, the server issues an access token (valid for 7 days) and a refresh token.

Role-Based Access Control (RBAC)

Role	Permissions
Admin	Full CRUD on all employee records, user management, system settings
HR Manager	Create/update employees, view all records, generate reports
Manager	View and update team members in own department
Employee	View and update own profile only

Token Flow

5. User submits credentials (email + password)
6. Server validates and returns JWT access token
7. Client stores token in httpOnly cookie
8. Each request includes Bearer token in Authorization header
9. Middleware verifies token and attaches user object to request

9. User Interface

The following screens make up the core UI of the Automating Employee Data application:

Dashboard

Figure 2: Main Dashboard with KPI cards and charts

Employee Directory

Figure 3: Employee Directory with search and filter

10. Testing

Testing Strategy

Test Type	Tool	Coverage Target
Unit Testing	Jest	Controllers, utility functions
Integration Testing	Supertest + Jest	API endpoints
Frontend Testing	React Testing Library + Jest	Components, user flows
E2E Testing	Cypress	Critical user journeys
Load Testing	Artillery	API performance under load

Running Tests

```
# Backend unit + integration tests
cd server && npm test

# Frontend component tests
cd client && npm test

# End-to-end tests
npx cypress open
```

11. Screenshots or Demo

Live Demo URL: https://automating-employee-data.vercel.app

Key Screenshots

- Login Page — secure JWT-based authentication form
- Employee Onboarding Wizard — multi-step form with validation
- Employee Profile View — full record with documents tab
- Analytics Dashboard — department-wise headcount, attrition charts
- Admin Panel — user management and system configuration

Figure 4: Analytics dashboard — headcount and attrition trends

12. Known Issues

Issue ID	Description	Severity	Status
BUG-001	Bulk CSV import fails for files > 5 MB	Medium	In Progress
BUG-002	Date picker does not localize to Tamil Nadu timezone (IST)	Low	Open
BUG-003	Session does not expire on mobile browsers after logout	High	In Progress
BUG-004	Export-to-PDF layout breaks on Safari 16	Low	Open

13. Future Enhancements

The following features are planned for future releases:

Priority	Feature	Target Version
High	AI-powered smart search across employee records	v1.1
High	Integration with biometric attendance systems	v1.1
Medium	Mobile app (React Native) for field managers	v1.2
Medium	Advanced leave and payroll automation module	v1.2
Low	Multi-language support (Tamil, Hindi)	v1.3
Low	Chatbot for HR queries using LLM integration	v2.0

End of Document

Automating Employee Data — v1.0 — 2025